

Capitolo 4

Work Package 4

4.1 Ambiente di simulazione

Il protocollo progettato in WP2 è stato realizzato come applicazione *stand-alone* in Python, strutturata in tre livelli: le primitive crittografiche (`src/crypto/`), le entità di protocollo (`src/entities/`) e un'interfaccia web Flask (`src/web/`) a fini dimostrativi. L'intera elezione viene eseguita in un singolo processo, senza richiedere né un'infrastruttura elettorale reale né un'applicazione mobile. Le entità del protocollo sono modellate come oggetti distinti che comunicano tramite chiamate di metodo e gli scambi di messaggi del protocollo corrispondono ai parametri passati a tali metodi. Tutte le misure di latenza sono state ottenute con il timer di Python; per ciascuna operazione si esegue un numero di ripetizioni indipendenti, riportando poi la media e deviazione standard. Le misure sono state raccolte su CPU **AMD Ryzen AI 7 350 w/ Radeon 860M (2.00 GHz)**, 32 GB di RAM, Python 3.10, sistema operativo Windows11, con 30 ripetizioni per operazione. La riproducibilità è garantita dallo script `benchmark.py` incluso nella repository.

4.1.1 Costo computazionale delle operazioni crittografiche

Operazione	1.000 schede	10.000 schede	100.000 schede
<i>Primitive Crittografiche di Base</i>			
RSA-2048 keygen (Setup)	35.753	37.541	38.483
OAEP encrypt (voto)	0.024	0.023	0.024
OAEP decrypt (spoglio)	0.254	0.251	0.260
RSA-PSS sign (IAP)	0.251	0.253	0.258
RSA-PSS verify (VBR)	0.025	0.026	0.025
Shamir split	0.009	0.010	0.010
Shamir reconstruct	0.042	0.042	0.043
Pub. decrypt check ($m^e = C$)	0.139	0.136	0.139
Trustee SHA-256 commitment	0.001	0.001	0.001
<i>Strutture Dati e Scalabilità</i>			
Merkle build	0.760	7.648	76.168
Merkle proof gen	0.002	0.005	0.014
Merkle proof verify	0.007	0.010	0.013

Tabella 4.1: Costo computazionale delle primitive (in ms) al variare del bacino elettorale.

La Tabella 4.1 riporta i **tempi di esecuzione medi** delle primitive crittografiche su cui poggia il protocollo. Per una corretta valutazione prestazionale, è opportuno distinguere le operazioni a complessità costante da quelle dipendenti dalla dimensione del bacino elettorale.

Le **primitive crittografiche di base** presentano un costo $O(1)$ rispetto al numero di elettori. Si osserva che la generazione della coppia di chiavi RSA-2048 costituisce l'operazione singola più onerosa (~ 38 ms), in quanto richiede la ricerca probabilistica di due numeri primi di grandi dimensioni. Tuttavia, collocandosi esclusivamente nella fase pre-elettorale di *Setup*, tale latenza non grava sul percorso critico del protocollo a regime. Le operazioni eseguite per ogni singola scheda durante la fase di sottomissione e scrutinio (cifatura/decifatura OAEP e firma/verifica RSA-PSS) manifestano un costo computazionale contenuto e dell'ordine di frazioni di millisecondo, garantendo un'elevata efficienza operativa.

Altrettanto efficiente risulta lo schema a soglia utilizzato per la frammentazione del segreto. La fase di ricostruzione della chiave, che richiede matematicamente di combinare un numero di frammenti pari alla soglia minima stabilita, si è dimostrata estremamente rapida. Avendo fissato una soglia operativa $t = 3$, l'algoritmo riesce a ricalcolare la chiave privata RSA in appena ~ 0.043 ms. Questo dato empirico conferma che la decentralizzazione della fiducia, scelta fondamentale per la sicurezza del protocollo discussa in sede di modello, non impatta in modo negativo sulle prestazioni e sulle tempistiche della fase pubblica di scrutinio.

Un'analisi separata merita l'**infrastruttura di tracciabilità** basata su Merkle Tree. Come si evince dai dati empirici raccolti su campioni da 1.000 a 100.000 foglie, è imperativo distinguere le complessità teoriche:

- **Costruzione del registro (Merkle Build):** Evidenzia una crescita lineare $O(N)$ coerente con la necessità di processare l'intero *dataset* di base. Il tempo passa da 0.760 ms per 1.000 votanti a 76.168 ms per 100.000 votanti. Si tratta di un *trade-off* accettabile, trattandosi di un'operazione eseguita dal VBR esclusivamente al consolidamento dell'urna.
- **Verificabilità (Proof Gen/Verify):** Le operazioni di estrazione e validazione delle prove di inclusione esibiscono, come atteso per gli alberi binari, una complessità logaritmica $O(\log_2 N)$. Anche nello scenario di stress con 100.000 elettori, la verifica individuale richiede latenze irrisorie (~ 0.013 ms), garantendo che l'ispezionabilità del sistema rimanga altamente scalabile e praticabile da qualsiasi dispositivo *client*.

4.1.2 Costo di Rete e Dimensione dei Messaggi

In ottemperanza ai requisiti di efficienza richiesti (WP1 e WP4), si è proceduto alla misurazione della dimensione effettiva dei messaggi scambiati tra le entità. Un protocollo di e-voting deve infatti garantire un *payload* sufficientemente leggero da poter essere trasmesso agevolmente anche su connessioni con banda limitata.

Elemento del Payload	1.000 schede	10.000 schede	100.000 schede
Token pseudo-casuale R (256-bit)	32 B	32 B	32 B
Firma IAP σ (RSA-PSS 2048)	256 B	256 B	256 B
Crittogramma C (RSA-OAEP 2048)	256 B	256 B	256 B
Sottomissione Voto (Dato grezzo)	544 B	544 B	544 B
Prova di Inclusione (Merkle Proof)	790 B	1106 B	1343 B

Tabella 4.2: Dimensione dei payload crittografici scambiati sul livello di rete.

I risultati (Tabella 4.2) confermano l'alta efficienza del protocollo. Il *payload* crittografico grezzo di una singola sottomissione di voto ha una dimensione fissa di soli 544 Byte, interamente determinata dalle lunghezze standard delle primitive scelte (le chiavi RSA da 2048 bit producono *ciphertext* e firme da 256 Byte, a cui si aggiungono i 32 Byte di entropia per il token R).

La tracciabilità logica tramite Albero di Merkle si riflette invece nella dimensione delle *Proof of Inclusion*. Poiché la prova richiede l'invio di un numero di nodi intermedi proporzionale all'altezza dell'albero logaritmico, la sua dimensione cresce asintoticamente di $\mathcal{O}(\log_2 N)$. Anche in presenza di uno scenario ad alto carico con 100.000 elettori, si mantiene il pacchetto di verifica al di sotto dei 1.5 KB. Questo dato dimostra empiricamente che la garanzia di verificabilità individuale scala senza introdurre colli di bottiglia.

4.1.3 Latenza delle Operazioni di Verifica

La verificabilità rappresenta uno dei pilastri fondamentali del protocollo Aequitas, garantita attraverso controlli crittografici distribuiti tra i diversi attori. Come discusso nel modello di minaccia, l'assenza di fiducia in un'unica entità centrale impone che ogni passaggio critico sia matematicamente validabile per prevenire alterazioni.

Operazione di Verifica	1.000 schede	10.000 schede	100.000 schede
Firma IAP su R (lato VBR)	0.025	0.026	0.027
Impegno Trustee (SHA-256)	0.001	0.001	0.001
Dec. Univ. $((m')^e \equiv C \pmod{N})$	0.131	0.140	0.142
Inclusione Merkle (lato Elettore)	0.007	0.010	0.012

Tabella 4.3: Latenza media (in *ms*) delle operazioni di verifica al variare del bacino elettorale.

Come si evince dalla Tabella 4.3, l'architettura implementa quattro distinti livelli di validazione, tre dei quali esibiscono una complessità costante $\mathcal{O}(1)$ indipendente dal bacino elettorale:

- **Verifica di Autenticità (VBR):** Durante la fase di voto, il VBR verifica la firma RSA-PSS dell'IAP apposta sul token. L'operazione richiede appena ~ 0.025 ms, confermando che il VBR può validare decine di migliaia di richieste al secondo, prevenendo colli di bottiglia.
- **Verifica degli Impegni (Tally Machine):** Prima della ricostruzione del segreto, viene verificato l'impegno di ogni fiduciario contro quelli pubblicati. Grazie all'efficienza computazionale delle funzioni di *hashing* crittografico, l'operazione si risolve in 0.001 ms.
- **Verificabilità Universale dello Spoglio:** Chiunque può accertare la corretta decifratura calcolando $(m')^e \bmod N = C$ sulle pre-immagini m' pubblicate dalla Tally Machine. Sfruttando l'esponente pubblico e , l'operazione risulta estremamente rapida (~ 0.14 ms per scheda), permettendo la validazione dell'intero scrutinio in pochi secondi, senza che la chiave privata venga esposta.

Infine, la **Verificabilità Individuale** è delegata alla prova di inclusione nell'albero di Merkle. Come discusso, il costo di questa operazione scala in modo logaritmico ($\mathcal{O}(\log_2 N)$). Un elettore che intenda verificare la presenza della propria ricevuta all'interno di un'urna contenente 100.000 schede impiegherà appena 0.012 ms. Questo dato conferma il pieno raggiungimento dell'obiettivo di progetto: offrire garanzie crittografiche solide e fruibili direttamente dai dispositivi *client* degli elettori, operando ampiamente al di sotto di qualsiasi latenza percettibile.

4.1.4 Latenze Macro-Architetturali e Scalabilità

Per valutare la scalabilità complessiva si è simulata un'elezione completa, misurando empiricamente le due macro-fasi infrastrutturali: il tempo di *freeze* del VBR (ordinamento crittografico dei record per garantire *unlinkability* e costruzione dell'albero di Merkle) e il tempo di *tally* (decifratura massiva di tutti i crittogrammi validi).

Fase Operativa	1.000 schede	10.000 schede	100.000 schede
Freeze VBR	2.00 ms	22.00 ms	234.01 ms
Tally Totale	253.96 ms	2598.25 ms	30546.49 ms
Tally singolo voto	0.254 ms	0.260 ms	0.305 ms

Tabella 4.4: Tempi di esecuzione misurati per consolidamento (*freeze*) e scrutinio (*tally*).

I risultati pratici confermano in pieno le previsioni teoriche (Tabella 4.4). Il tempo di *tally* cresce in modo lineare $\mathcal{O}(N)$, dato che la decifratura di ogni scheda è un'operazione indipendente. Il tempo richiesto per ogni singolo voto si mantiene stabile tra 0.26 e 0.30 ms; questo permette uno spoglio rapidissimo, capace di decifrare un'urna intera da 100.000 schede in circa 30 secondi utilizzando un processore standard. Anche il tempo di *freeze* si dimostra estremamente efficiente (~ 234 millisecondi nel caso di carico massimo).

Si conclude quindi che il sistema non soffre di alcun collo di bottiglia legato alla crittografia. L'unica vera "attesa" percepita dall'utente durante il voto è dovuta esclusivamente al ritardo casuale δ (ad es. 2-5 secondi) inserito di proposito dall'applicazione *client*. Come discusso nel WP3, questo ritardo non è un limite tecnico, ma una precisa e indispensabile scelta di sicurezza: serve a proteggere lo pseudo-anonimato dell'elettore sventando gli attacchi basati sull'analisi del traffico di rete (*Traffic Analysis*).